

GIT OPERATIONS & COMMANDS

A practical beginner's guide to Git and GitHub

Learn the everyday workflow: create a repository, track changes, commit work, collaborate through branches, publish to GitHub, and recover from common mistakes.

Based on: W3Schools Git tutorial (GitHub learning path), with practical examples and safety notes.

Contents

- 1. Git and GitHub fundamentals
- 2. The Git workflow
- 3. Installation and configuration
- 4. Create or clone a repository
- 5. Stage, commit, and inspect changes
- 6. Connect to GitHub and push
- 7. Branches and merging
- 8. Fetch, pull, and remote branches
- 9. Merge conflicts
- 10. Undoing changes safely
- 11. Stash, .gitignore, files, and tags
- 12. Command quick reference
- 13. Complete practical workflow
- 14. Common errors and best practices
- 15. Practice checklist and learning resources

1. Git and GitHub fundamentals

Git is a version control system. It records how files change over time so you can review earlier work, compare versions, and collaborate.

GitHub is an online platform for hosting Git repositories and collaborating with others. Git can work locally without an internet connection; GitHub is used to share and host repositories.

Term	Meaning
Repository (repo)	A project managed by Git.
Local repository	A Git project on your computer.
Remote repository	A repository hosted elsewhere, such as GitHub.
Commit	A saved checkpoint in the project's history.
Branch	A separate line of development.
Staging area	The place where you prepare changes for the next commit.
Push / Pull	Send local commits / fetch and integrate remote changes.
Clone	Copy an existing repository to your computer.
Merge	Combine work from another branch.
HEAD	Your current position in Git history.
.gitignore	Rules for files Git should ignore.

2. The Git workflow

Git commonly involves three local areas. You edit in the working directory, select changes for the staging area, then commit them to local history. Push sends committed work to GitHub.

Stage	What it means	Command
Working directory	Edit, add, or delete files	Your editor / file system
Staging area	Choose changes for the next commit	git add
Local repository	Save a checkpoint in history	git commit
Remote repository	Share commits on GitHub	git push

Remember: `git add` and `git commit` do not upload anything to GitHub.

3. Install and configure Git

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

Install Git from git-scm.com/downloads. The global settings apply to your user account across repositories. Without `--global`, settings can be repository-specific.

4. Create or clone a repository

Start a new local project

```
mkdir my-first-project
cd my-first-project
git init
git status
```

`git init` creates the hidden `.git` data directory and starts a repository in the current folder.

Get an existing project

```
git clone https://github.com/username/project.git
cd project
git remote -v
```

Cloning downloads the files and history and normally configures the source remote as `origin`. Use `git init` for a new repository; use `git clone` for an existing one.

5. Stage, commit, and inspect

```
git status
git add README.md
git add .
git diff
git diff --staged
git commit -m "Add project README"
git log --oneline
```

`git add file` stages one file. `git add .` stages changes under the current directory; review first so you do not include unintended files. `git diff` shows unstaged changes; `git diff --staged` shows what is prepared for commit.

Use short, descriptive messages such as `Fix login validation` or `Add homepage layout`.

6. Connect to GitHub and push

```
git remote add origin https://github.com/username/my-first-project.git
git remote -v
git branch
git push -u origin main
```

Create an empty repository on GitHub, then add its URL as a remote. `origin` is a conventional remote name, not a special command. Replace `main` with your actual branch if it is named differently.

The `-u` option sets upstream tracking. After the first push, you can often use `git push`. GitHub HTTPS authentication uses a credential manager or token; your normal account password is not accepted as the Git password.

7. Branches and merging

A branch lets you develop a feature separately from the main line of work. Create a feature branch instead of making every change directly on `main`.

```
git branch
git branch feature-login
git switch feature-login
git switch -c feature-login
git switch main
git merge feature-login
git branch -d feature-login
```

`git switch -c` creates and switches in one step. Merge while on the branch that should receive the work (for example, switch to `main`, then merge the feature branch). `-d` deletes a merged local branch; `-D` force-deletes and may discard unmerged work.

```
git branch -m old-name new-name
git push -u origin feature-login
git push origin --delete feature-login
```

The first command renames a local branch. The latter commands publish or delete a remote branch.

8. Fetch, pull, and remote branches

Command	Meaning
<code>git fetch origin</code>	Download remote updates without integrating them into the current branch.
<code>git pull</code>	Fetch and integrate remote changes into the current branch.
<code>git pull origin main</code>	Integrate the remote main branch.
<code>git branch -r</code>	List remote-tracking branches.
<code>git branch -a</code>	List local and remote-tracking branches.
<code>git fetch --prune</code>	Fetch updates and remove stale remote-tracking references.

Use `fetch` when you want to inspect incoming changes before integrating them. Use `pull` when you are ready to update your current branch.

9. Merge conflicts

A conflict occurs when Git cannot automatically combine competing edits, often because two branches changed the same part of a file. Git marks the conflicting area:

```
<<<<<< HEAD
const message = "Hello World";
=====
const message = "Welcome to Git";
>>>>>> feature-login
```

Open the file, decide what the final code should be, remove the conflict markers, and save. Then stage the resolved file and complete the merge:

```
git status
git add filename.js
git commit -m "Resolve merge conflict"
```

To cancel an in-progress merge instead, use `git merge --abort`. A conflict is not necessarily a code error; Git needs a human decision.

10. Undoing changes safely

Command	Effect
<code>git restore file.js</code>	Discard unstaged edits in the file.
<code>git restore --staged file.js</code>	Unstage the file but keep its edits.
<code>git restore --staged .</code>	Unstage all staged changes.
<code>git reset --soft HEAD~1</code>	Undo last commit; keep changes staged.
<code>git reset HEAD~1</code>	Undo last commit; keep changes unstaged.
<code>git reset --hard HEAD~1</code>	Move back one commit and reset tracked files; destructive.
<code>git revert HASH</code>	Create a new commit that reverses a prior commit.

Safety rule: `restore` can discard edits, and `reset --hard` can destroy uncommitted work. For a commit already shared with others, prefer `git revert` rather than rewriting shared history with `reset`.

11. Stash, ignore rules, files, and tags

Temporarily save unfinished work

```
git stash push -m "Work in progress"
git stash list
git stash apply
git stash pop
git stash drop stash@{0}
git stash clear
```

`apply` restores a stash but keeps it; `pop` restores and removes it if successful. Stash primarily captures tracked changes; use options such as `-u` if you also need untracked files.

Ignore files

```
# Example .gitignore
.env
.env.local
node_modules/
dist/
build/
*.log
.DS_Store
Thumbs.db
```

A `.gitignore` rule does not stop tracking a file already tracked by Git. To keep a local file but remove it from Git tracking:

```
git rm --cached .env
git commit -m "Stop tracking environment file"
```

Never commit secrets. If a key or password was committed, treat it as exposed and rotate it; deleting the file alone is not enough.

Remove and rename files

```
git rm filename.js
git rm --cached filename.js
git mv old-name.js new-name.js
```

`git rm` removes and stages deletion. `--cached` keeps the local file but stops tracking it. `git mv` renames and stages the operation.

Tags

```
git tag -a v1.0.0 -m "First stable release"
git tag
git push origin v1.0.0
git push origin --tags
git tag -d v1.0.0
git push origin --delete v1.0.0
```

Tags label specific commits, often for releases. Annotated tags include a message and tagger information.

12. Command quick reference

Task	Command
Check status / version	git status git --version
Initialize / clone	git init git clone URL
Stage / commit	git add file git commit -m "message"
Review changes	git diff git diff --staged
History / commit detail	git log --oneline git show HASH
Branches	git branch git switch -c name git switch main
Merge	git merge branch-name
Remote	git remote -v git remote add origin URL
Share / update	git push git pull git fetch
Undo	git restore file git revert HASH
Stash	git stash git stash list git stash pop
Cleanup preview	git clean -n
Recent reference movements	git reflog

Destructive cleanup: `git clean -n` previews untracked files that would be removed. `git clean -f` deletes them. Preview first.

13. Complete practical workflow

Scenario: you cloned a web application and need to add a login feature.

```
git clone https://github.com/username/my-project.git
cd my-project
git switch main
git pull origin main
git switch -c feature/login
# Make and test your code changes
git status
git diff
git add src/components/Login.jsx
git diff --staged
git commit -m "Add login functionality"
git push -u origin feature/login
```

Then open GitHub, create a Pull Request from `feature/login` into `main`, describe the changes, request review, and merge after approval. Finally update local `main`:

```
git switch main
git pull origin main
git branch -d feature/login
```

Delete the remote `feature` branch if GitHub has not already done so:

```
git push origin --delete feature/login
```

14. Common errors and best practices

Message / problem	Likely cause and response
fatal: not a git repository	You are outside the repository. Navigate to the project folder.
nothing to commit	No staged changes. Check git status.
src refspec main does not match any	Branch may have another name, or no commit exists. Check git branch and commit.
remote origin already exists	A remote is already configured. Inspect git remote -v.
non-fast-forward push rejected	Remote has commits you do not have. Fetch and integrate before pushing.
merge conflict	Edit conflict markers, stage resolved files, and complete the merge.
authentication failed	Check your credential manager, token, or SSH setup.

If a push is rejected because the remote contains newer work, one common workflow is:

```
git fetch origin
git pull --rebase origin main
# Resolve any conflicts, then:
git add filename.js
git rebase --continue
git push origin main
```

Cancel a rebase with `git rebase --abort`. Avoid `git push --force` as a quick fix; it can overwrite remote history.

Best practices

- Check your branch before editing: `git branch`.
- Run `git status` often.
- Make small commits that represent one logical change.
- Write descriptive commit messages.
- Review staged changes before committing.
- Use feature branches and Pull Requests for collaboration.
- Keep secrets and local configuration out of Git.
- Prefer revert for undoing commits already shared.
- Understand destructive commands before running them.

15. Practice checklist

Exercise 1 - Initialize

- Create a folder called `git-practice`.
- Run `git init` and check status.
- Create `README.md`.

Exercise 2 - First commit

- Stage `README.md` and commit it.
- View history with `git log --oneline`.

- Edit the file and inspect git diff.

Exercise 3 - Branching

- Create and switch to feature/about.
- Create about.txt and commit it.
- Switch back to main.

Exercise 4 - Merging

- Merge feature/about into main.
- Verify the file exists and inspect history.

Exercise 5 - GitHub

- Create a GitHub repository.
- Add it as origin and push main.
- Verify the files online.

Exercise 6 - Undo and stash

- Try git restore on an unstaged edit.
- Stage and unstage a change.
- Stash a change and restore it.

Further learning

- W3Schools Git tutorial: <https://www.w3schools.com/git/default.asp?remote=github>
- Official Git documentation: <https://git-scm.com/docs>
- Learn Git Branching (interactive): <https://learngitbranching.js.org/>
- GitHub Getting Started: <https://docs.github.com/en/get-started>

Learning tip: Practise in a throwaway repository first, especially before using reset, clean, or force-push commands.